

Spring 2025, Raft Assignment

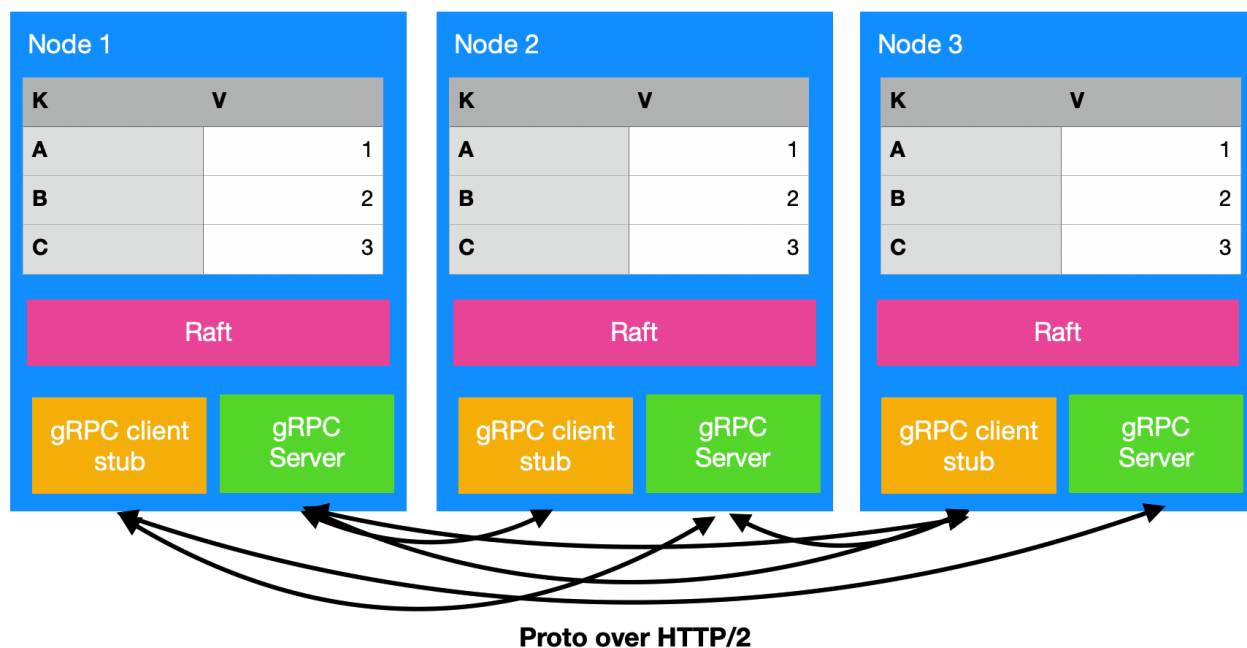
The Chinese University of Hong Kong

Deadline: April 8, 2026, 11:59 am

Late Policy: 30% discount for any late. Final Deadline: Apr 16, 2026, 11:59am

Contents

1	Introduction	2
2	Your Assignment	5
2.1	Submission: git classroom	5
2.2	Get the assignment package	5
2.3	Get your starter code based on your preferred language	7
2.4	Submission	7
2.5	Your job	8
3	Run our grader	9
4	Grading	11
5	Remarks and hints	11
6	Change Log	13
7	Questions	13
8	General Notes	14



1 Introduction

In this assignment, you will build a **distributed replicated (key-value) map** using Raft [1]. Follow the Raft paper to implement and pass all the test cases in the grader. We thank Shimin Wang from Carnegie Mellon University when preparing this assignment.

For the details of Raft please refer to

- The details of Raft presented by their authors: <https://youtu.be/YbZ3zDzDnrw> and <https://youtu.be/vYp4LYbnnW8>
- The Raft extended paper annotated by Shimin Wang, which is included in the assignment package.
- The official web site of Raft, <http://raft.github.io>. It provides a good interactive demo with the Raft consensus protocol.

You do not need to handle all cases in the full Raft paper. Specifically,

- You don't need to randomize the election timeout. The election timeout value would be changed by the grader at run-time to simulate different situations.

- You don't need to implement log compaction and snapshotting.
- We don't need to handle cluster membership changes.
- No disk is considered in this implementation. Your key-value map is in memory. When you commit any log, you should directly apply that operation, committed will be treated equally as applied. Consequently, the `lastApplied` operation in the Raft paper is unnecessary to you.

Name	Description
<code>bin/</code>	Student-compiled binaries and grading tools binary.
<code>scripts/</code>	Grading shell scripts (Don't touch).
<code>tests/</code>	Source code of grading tools. (Don't touch).
<code>yourCode/</code>	Your working directory
<code>yourCode/main.go</code>	Write all your code in this file (Work on it).
<code>yourCode/compile.sh</code>	Script to compile your program (Don't touch it).
<code>raft.proto</code>	The gRPC proto file. (Don't touch . Just FYI only, since we have already generated the stubs for you.)
<code>raft(annotated).pdf</code>	The extended raft paper with some hints for the implementation (Please read it).

Remember to include both Src and Dst node id in both args and replies when you handle both RequestVote RPC and AppendEntries RPC!!

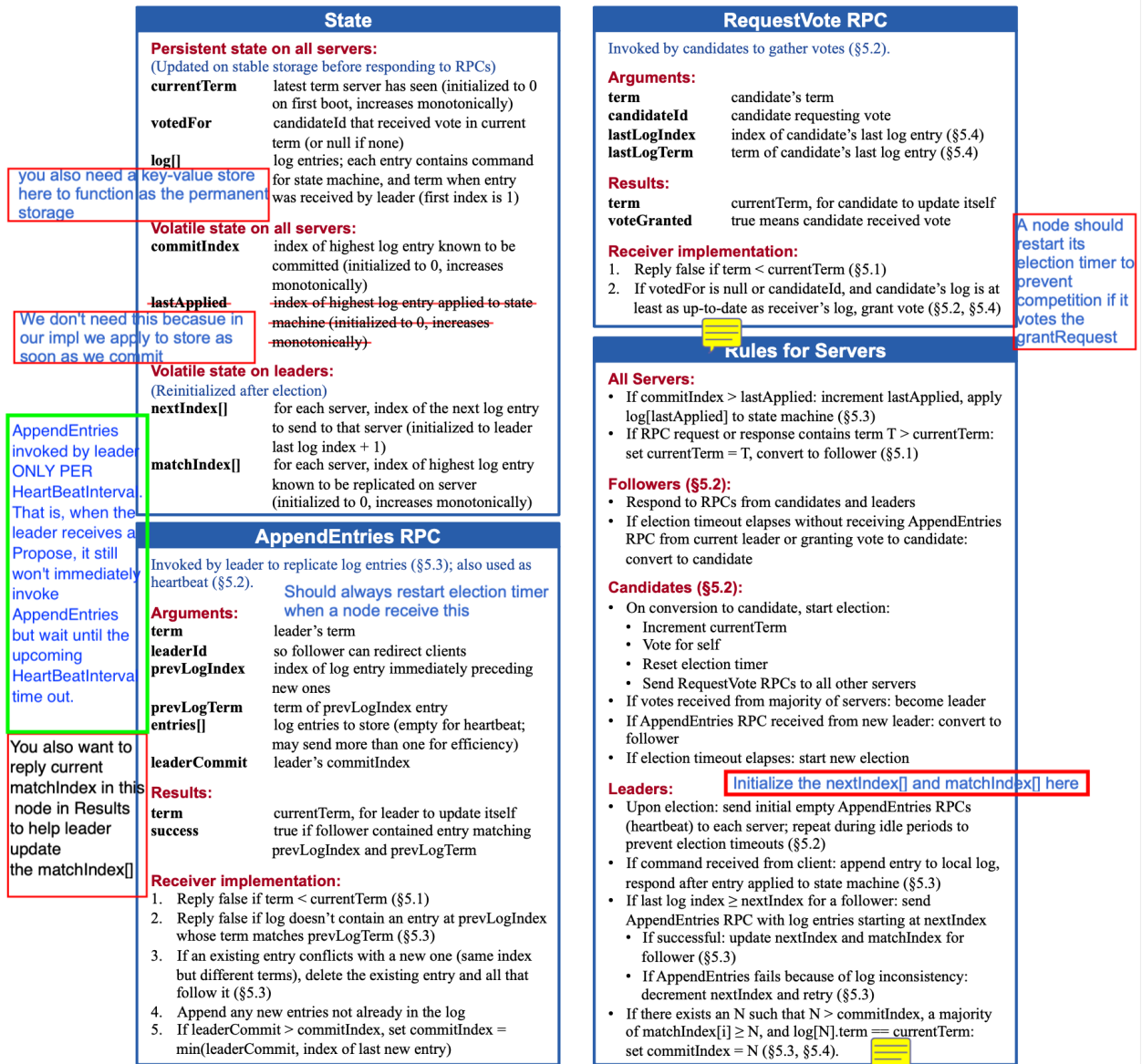


Figure 1: A condensed summary of the Raft consensus algorithm

2 Your Assignment

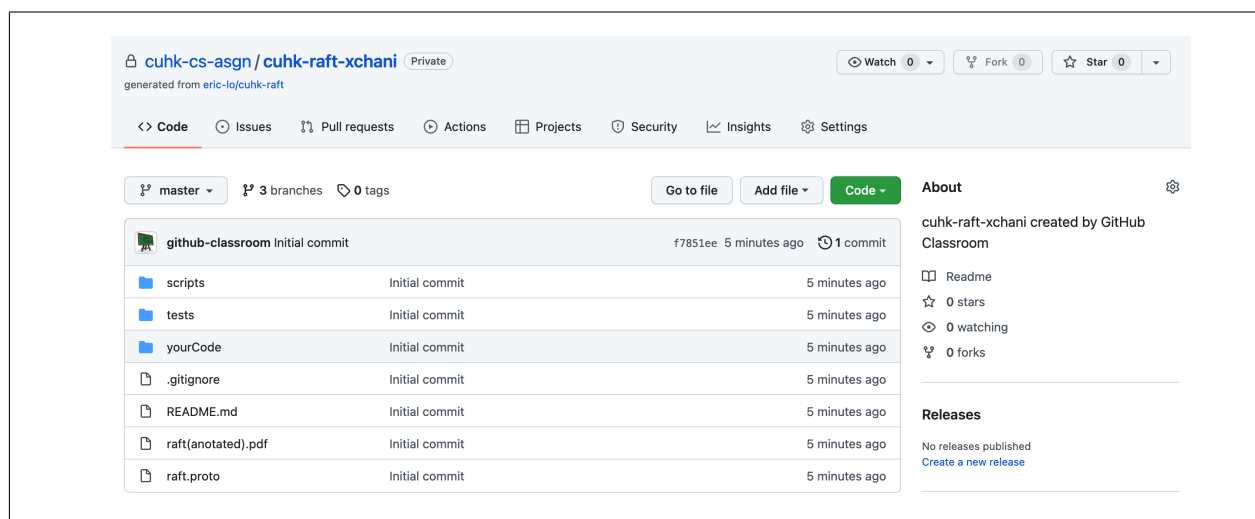
2.1 Submission: git classroom

We use **GitHub Classroom** <https://classroom.github.com> for assignment management. Git is a predominant version control system, created by the inventor of Linux. GitHub is an online hosting of Git and it is now hosting thousands of open-source projects.

2.2 Get the assignment package

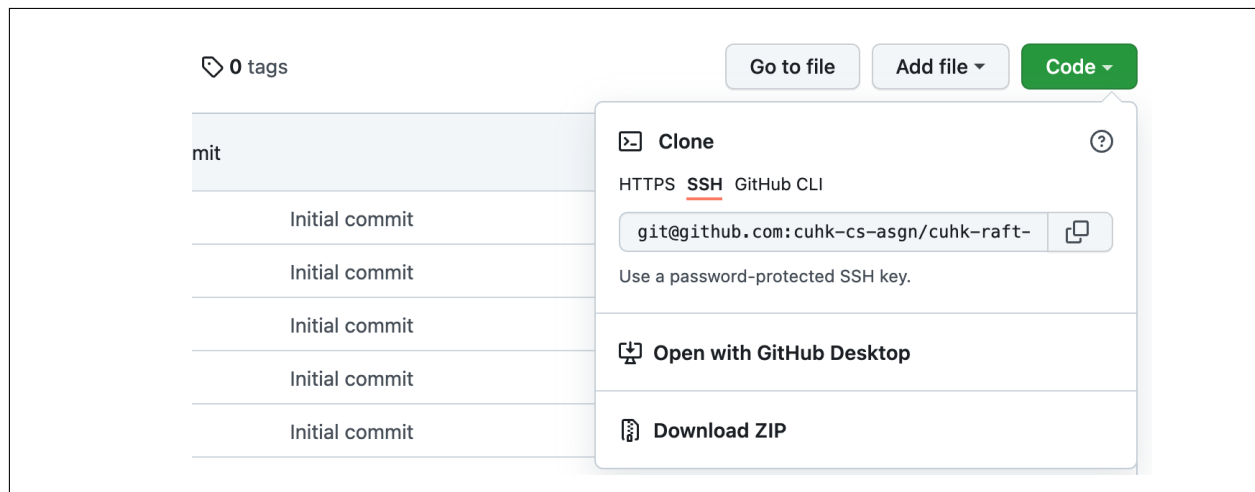
You are supposed to open a Github account using your CUHK email address (it is fine to use your own Github account if you already have one) and follow the steps below to register an account and get the starter code.

1. Use your Github account to come here <https://classroom.github.com/a/w5BDTe7G> to clone the assignment starter package to be under your Github account.
2. If everything runs smoothly, your Github account will show a new repo named `cuhk-raft-[your-Github-account-name]`, like below:



3. Sometimes you cannot access github repository immediately after accepted classroom link. If you cannot access, please wait for a while and you should receive an email from github-classroom[bot]. Accept the invitation and you can access your assignment repository.

4. As GitHub shut down password authentication in Aug. 2021, you're required to setup an ssh key for connection following the GitHub Docs.
5. Look for the tab "Code" to get the URL of your assignment repo, like below:



6. Open a terminal, change to a local directory of your desire, and type "git clone {Your Repo URL}" and your username/password to get a local copy of the repo on your computer. Also, feel free to use **HTTPS** or **GitHub CLI** way to clone the repo. *Note that when you choose the **GitHub CLI** way, you may have to run command "git remote remove upstream" to firstly remove the tracked original repo.*

```
→ Github git clone git@github.com:cuhk-cs-asgn/cuhk-raft-xchani.git
Cloning into 'cuhk-raft-xchani'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 42 (delta 6), reused 21 (delta 1), pack-reused 0
Receiving objects: 100% (42/42), 714.57 KiB | 603.00 KiB/s, done.
Resolving deltas: 100% (6/6), done.
```

7. Finally, we should set the user name and email such that the git system recognizes you. You should do this once only. To begin with, type the followings in the terminal and **replace the name and email** with your own Git account.

```
→ cuhk-raft-xchani git:(master) git config --global user.name "Chenxia Han"
→ cuhk-raft-xchani git:(master) git config --global user.email "cxhan@cse.cuhk.edu.hk"
```

2.3 Get your starter code based on your preferred language

First, clone the project from your repository. Please execute

```
git checkout go
```

2.4 Submission

Git is actually a complicated version control system (VCS) that has a lot of features. Briefly, it installs a local VCS on your computer and you can do any versioning locally (e.g., committing some changes, reverting some changes). If necessary, you can “push” your local changes from your local repo to an online repo (Github). You may Google the powerful usage of Git online. Here, we focus on how you can commit your finished assignment to your local repo and then push to the online private repo. We will fetch your assignment from your online private repo for grading. The process is essentially the “assignment submission” process:

```
→ cuhk-raft-xchani git:(go) ✗ git add .
→ cuhk-raft-xchani git:(go) ✗ git commit -m "Test commit"
[go 804ac0a] Test commit
 1 file changed, 1 insertion(+)
→ cuhk-raft-xchani git:(go) git push origin
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 349 bytes | 349.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:cuhk-cs-asgn/cuhk-raft-xchani.git
 36d8001..804ac0a  go -> go
```

Assuming you have done some edits on the source files, the command `git add .` states your interest of adding everything under the current directory “.” to your local Git repo. Next, the command `git commit` tries to commit your changes to your local repo. The

parameter `-m "comments"` is your comment about this commit. Lastly, the command `git push origin` tries to synchronize your local repo with your online repo.

For Git beginners, make sure you use a web browser to login your Github online and check if your source codes have been pushed there successfully.

To submit the final version for grading, push the code to the `master` branch by `git push origin go:master -f`.

Warning: DON'T change the file names, otherwise you get 0 marks

2.5 Your job

Write your program in `'main.go'`.

- Follow the “State” box in Figure 1 to implement `raftNode` struct (Go). You will need to add a lot more than what Figure 1 tells you such as adding a `Map` to be the in-memory key-value map, adding a `server-state` to store whether this node is in “Follower”, “Candidate”, or “Leader” state, etc.
- Function `NewRaftNode`: After creating an instance of `raftNode`, its initial state should be a “Follower”, a node shall create a new thread to kick off a new leader election if it has not heard from the leader within a period of `electionTimeout`. To kick off the leader election, invoke the others’ `RequestVote` by RPC **concurrently**. If a remote node doesn’t respond to your `RequestVote`, don’t resend `RequestVote`, wait for the next election.
- When a node becomes a “leader”, invoke others’ `AppendEntries` RPC concurrently based on the heartbeat interval.

Follow Figure 1’s “AppendEntries” box to implement it. Once again, if a remote node doesn’t respond to your call, don’t resend, wait for the next heartbeat interval.

- RPC `Propose`: Our grader will call this RPC to `put` or `delete` to update the distributed replicated map.

Log this operation using `LogEntry` and append to the local log of the leader. `Propose` returns only after the log is committed (i.e., replicated to the majority of other nodes).

- RPC `GetValue`: Implement this to return a value given a key.
- RPCs `SetElectionTimeout` and `SetHeartBeatInterval`: invoked by the grader to set up different test cases. On invoked, these functions (re)-set the election timeout and heartbeat interval based on the input values.
- **NOTE:** When implementing by using **Go**, you are prohibited from using **mutex** and related components. **Otherwise**, you will receive a score of **0**. Use **channel** to implement the relevant functions.

After you have finished the assignment, compile it first to check whether there is any syntax error. You shall first enter into `yourCode` and execute:

```
bash compile.sh
```

A successful compilation would generate the `bin/raftrunner` file.

3 Run our grader

After your program can be successfully compiled, run `scripts/rafttest.sh` to compile the test framework and check whether you can pass the given test cases. The grader will launch 5 Raft instances as 5 processes.¹ There are 10 test cases: 6 to test the leader election, and 4 to test the log replication. For each test case, it may

- Set different election timeouts and a heartbeat interval on each node (process)
- Delay or drop the messages between the nodes
- Invoke `Propose` RPC to update the distributed replicated map
- Invoke `GetValue` RPC to get value by a key and match it with an expected value
- Match the messages between different nodes with the expected messages
- Note: there is a running time limit for each test case

¹It will also launch 5 proxies for the raft instances. The proxies are used by the grader to delay the messages (to simulate various test cases), etc. You don't need to understand them.

All tests run about 200 ~ 300 seconds. If a test case fails, the reason and the expected messages (if any) will be printed out. For example:

```
Expected:
node 0 <- 2: RequestVote -- term: 1, candidateId: 2, lastLogIdx: 0, lastLogTerm: 0
node 0 -> 2: reject, term: 1
node 0 <- 2: AppendEntries -- term: 1, leaderId: 2, prevLogIdx: 0, prevLogTerm: 0, entries: [], leaderCommit: 0
node 0 -> 2: success, term: 1, matchIdx: 0
But get:
node 0 <- 1: RequestVote -- term: 2, candidateId: 1, lastLogIdx: 0, lastLogTerm: 0
node 0 <- 2: RequestVote -- term: 2, candidateId: 2, lastLogIdx: 0, lastLogTerm: 0
node 0 -> 1: granted, term: 2
node 0 -> 2: reject, term: 2
testTwoCandidateForElection FAIL
```

In the above, the message “node 0 <- 2: RequestVote -- term: 1, ...” means node 2 requests a vote from node 0, and the one who sends the message (node 2) is in term 1. Similarly, the message “node 0 -> 2: reject, term: 1” means node 0 replies node 2 with a reject, and the one who sends the message (node 0) is in term 1.

In the end of the test, you will see a summary. If you see something like below, congratulations.

```
2022/01/14 03:13:01 testOneCandidateOneRoundElection PASS
2022/01/14 03:13:05 testOneCandidateStartTwoElection PASS
2022/01/14 03:13:11 testTwoCandidateForElection PASS
2022/01/14 03:13:15 testSplitVote PASS
2022/01/14 03:13:19 testAllForElection PASS
2022/01/14 03:13:23 testLeaderRevertToFollower PASS
2022/01/14 03:13:28 testOneSimplePut PASS
2022/01/14 03:13:34 testOneSimpleUpdate PASS
2022/01/14 03:13:41 testOneSimpleDelete PASS
2022/01/14 03:13:48 testDeleteNonExistKey PASS
```

We also provide a script to test for a specific test case. For example:

```
bash ../scripts/rafttest_single.sh \
    testOneCandidateOneRoundElection
```

Remember to run `rafttest.sh` at least once before `rafttest_single.sh`, otherwise, the grader is not compiled and can't be found.

4 Grading

1. The TA will fetch and grade your latest version of **master** branch in your repository as of the deadline. **Remember to commit and push before the deadline!**
2. 10 test cases in total.
3. Passing one test case will earn you 10 marks.
4. A test case is regarded as PASS if and only if your program always pass it. If it fails once during the grading, it's regarded as FAIL.
5. Part A is leader election part of Raft. It covers the first six test cases. Part B is the log replication part, which covers the last four test cases.
6. We'll try to run the grader as many times as we can. As far as we know, 10 times is not enough to find all bugs, please run the grader for more times to fully test your program before submission.

5 Remarks and hints

- Don't deadline fighting! We give you such a long deadline for a reason – this assignment might firstly take you some weeks to start passing the first test case. Some more weeks to pass some more but may take even more weeks to pass only one more (and some previously passed test cases might fail after you pass a new one! i.e., regression). The above cycle might repeat! Simply put, start early.
- In our experience, sometimes you may spend hours debugging just 1 line of code (e.g., changing its position). Yes, it is normal (and painful). But we all ran through it.
- Apart from the already imported packages, only the standard library is allowed to use.
- Work on leader election first (to pass the first 6 test cases). By that time, you may ignore those log replication related entities like `commitIndex` and work back on those when you start working on log replication.

- Implement `SetElectionTimeout` and `SetHeartBeatInterval` first. They are not built-in Raft RPCs but used by the grader to simulate different test cases.
- Don't use Go's `time.Timer` or `time.Ticker`, which are difficult to use correctly.
- The parameters `heartBeatInterval` and `electionTimeout` in the `NewRaftNode` function are the default values for heartbeat interval and election timeout.
- For every node, set `lastLogIndex` = 0 and `lastLogTerm` = 0 initially.
- When winning an election, immediately announce winner by `AppendEntries`(empty log entry), don't wait for next heart beat.
- On winning the election, the leader creates 1 thread per follower and reuses those threads until the leader dies.
- Remember to update `votedFor` after you vote for a candidate.
- Set 100ms connection timeout when contacting a follower as the grader would sometimes emulate messages dropped.
- The log is implemented as an array. But to follow the Raft paper, our test regards the array begins from index **1** (i.e., `log[1]` is the first element), not from **0** (i.e., no `log[0]`).
- In this version of Raft, every follower shall also answer to `GetValue` from its own key-value store, not redirecting that request to the leader.
- If a `Propose` is sent to a follower node, `Status_WrongNode` shall be replied to tell the requester who is the current leader and the requester shall resend the request to the current leader.
- For the delete operation, you have to check whether the key exists in the leader's key-value store. The timing is very tricky: you have to wait for the majority of followers replied (i.e., the log entry of that operation is committed) before you really delete the key from the key-value store.

- There is certain randomness in distributed systems. So, if you pass a test case, make sure you repeat the test as many times as you can because sometimes you may just pass it by accident.
- Run `rafttest.sh` frequently to make sure you won't break previous passed test cases when working on a new one.
- It's possible that the grader doesn't kill the processes completely. If the grader reports that some ports are occupied by other processes, type `"ps aux | grep raft"` to check and `"kill -9 $(pgrep -f raft)"` to kill those processes.
- Check `tests/rafttest/leaderElectionTest.go` and `logReplicationTest.go` to figure out the test cases if necessary.
- When an RPC is called by another process, gRPC would auto launch a new thread to execute that RPC function.
- Ignore those `XXX_` entries in the RPC.

6 Change Log

1.0	this document
-----	---------------

7 Questions

If you have doubts about the assignment, you are encouraged to ask questions on Blackboard **under the corresponding discussion**. Please focus on knowledge. Unhealthy questions/comments that focus on scores and grades are not encouraged.

If you find any (possible) bugs, send private messages on Blackboard to us instead — otherwise that may cause unnecessary panic among the class if that is not a real bug.

8 General Notes

- Using AI coding tools is NOT allowed.
- If you use your own environment except for the environment specified by specification and got any question, test it on our specified environment before you ask.
- The TA reserves the right to adjust your scores for any request that requires their extra manual effort.
- Unless specified, you should work and fill your code in designated areas. There are cases that the modification outside the designated area leads to misbehavior of the auto grader. Proceed with caution and look back the changes if your output is different from what is shown in the grader.
- While we have already tried our best to prepare this assignment, we reserve all the rights to update the specification and the grading scheme. If there are any mistakes/bugs which are on ours, the TA will step in and do manual grading to ensure you get what you deserve. Please respect each other.
- When grading, the TAs will execute the grader program to run the whole test suite (which consists of all test cases). The TAs won't grade each individual test case separately.
- **(Frequently Asked)** [No hidden test case] We are not intended to run any secret/extra test cases that deliberately break your assignment. That is, WYSIWYG — your final score shall be generally indicated by what the grader reports when it runs on our the environment specified by the specification. However, we do reserve the right to run some additional test cases to avoid any mis-conduct (e.g., hard-coding the results), and/or invite you to explain the source code to the teaching assistants and adjust the scores accordingly.
- We welcome discussions among classmates. But don't share your assignment with the others in any means. For example, don't put your source code in any public venue (e.g, public repo, your homepage, Facebook). On submitting this assignment, you are

agreed that your code's copyright belongs to the Chinese University of Hong Kong. Please do **not** release your source code and this specification online.

- Google is your friend. We encourage you use Google for help to do the assignment. However, if you happen to find any source codes related to this assignment, you still cannot copy it but use your own way to implement it. You need to put down your list of source code references as comments in the top of your source code.

References

- [1] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In 2014 {USENIX} Annual Technical Conference ({USENIX}{ATC} 14), pages 305–319, 2014.